Game Server Algorithms

Ap Calderon, Calista Ruiz, Erick Rayas, Linh Tran, Raul Munoz

Group 8

Story

We have just created a game and are expecting our servers to be a capacity often so we must work on a queue system for users waiting to enter the game.

- Game servers are a major part of hosting multiple players during an online game session.
- Servers are limited in player space
  - Exceeding space can result in problems such as overloading and lagging in-game.
- Algorithms to determine user connection:
  - First Come First Serve
  - Shortest Job First
  - Priority Scheduling.

Algorithms

First Come First Serve:

- The most common method used as users will join the server queue in the order of which they first joined.

Priority Scheduling:

- The second most common method which incorporates a premium option where the user will pay a monthly subscription to be prioritized by the server and join ahead of most players.

Shortest Job First:

- Not as common but users’ connection to the server is dependent on which players have the faster connection times so the queue is cleared as fast as possible, maximizing throughput

Initial Input Design

```
Server started at 00:00

  00:00   User1 [Member] has joined the server queue
  00:01   User2 [Member] has joined the server queue
  00:02   User3 [VIP] has joined the server queue
  00:04   User4 [Member] has joined the server queue
  00:06   User5 [VIP] has joined the server queue


Press Enter to run simulations...
```

Process (Unique and Individual Player)	**Arrival** (Start establishing a connection to the server)	**Burst** (Time it takes for the server to authenticate the user)	**Priority** (Premium users who are given access to the front of the line to join the server)
User1 | [Free]	0	6	2
User2 | [Free]	1	3	2
User3 | [Priority]	2	5	1
User4 | [Free]	4	1	2
User5 | [Priority]	6	4	1

```
--- First Come First Serve ---

  00:06  User1      [Member]   left after waiting   0s (server processed in  6s) | TAT:   6s | loaded at: 00:06
  00:09  User2      [Member]   left after waiting   5s (server processed in  3s) | TAT:   8s | loaded at: 00:09
  00:14  User3      [VIP]      left after waiting   7s (server processed in  5s) | TAT:  12s | loaded at: 00:14
  00:15  User4      [Member]   left after waiting  10s (server processed in  1s) | TAT:  11s | loaded at: 00:15
  00:19  User5      [VIP]      left after waiting   9s (server processed in  4s) | TAT:  13s | loaded at: 00:19

  Avg Wait Time: 6.2s  |  Avg Turnaround Time: 10.0s  |  Avg Completion Time: 12.6s
```

Process (Unique User)	**Arrival**	**Burst**	**CT**	**WT: TAT-BT**	**TAT: CT-AT**
User1 | [Free]	0	6	6 [6+0]	0	6
User2 | [Free]	1	3	9 [6+3]	5	8
User3 | [Priority]	2	5	14 [9+5]	7	12
User4 | [Free]	4	1	15 [14+1]	10	11
User5 | [Priority]	6	4	19 [15+4]	9	13

Average WT: (0+5+7+10+9 = 31/5 = **6.2**) | **Average TAT**: (6+8+12+11+13 = 50/5 = **10**)

Order of completion:

1 -> 2 -> 3 -> 4 -> 5

```
--- Priority Scheduling ---

  00:06  User1      [Member]   left after waiting   0s (server processed in  6s) | TAT:   6s | loaded at: 00:06
  00:18  User2      [Member]   left after waiting  14s (server processed in  3s) | TAT:  17s | loaded at: 00:18
  00:11  User3      [VIP]      left after waiting   4s (server processed in  5s) | TAT:   9s | loaded at: 00:11
  00:19  User4      [Member]   left after waiting  14s (server processed in  1s) | TAT:  15s | loaded at: 00:19
  00:15  User5      [VIP]      left after waiting   5s (server processed in  4s) | TAT:   9s | loaded at: 00:15

  Avg Wait Time: 7.4s  |  Avg Turnaround Time: 11.2s  |  Avg Completion Time: 13.8s
```

Process (Unique User)	**Arrival**	**Burst**	**Priority**	**CT**	**WT: TAT-BT**	**TAT: CT-AT**
User1 | [Free]	0	6	2	6 [6+0]	0	6
User2 | [Free]	1	3	2	18 [15+3]	14	17
User3 | [Priority]	2	5	1	11 [6+5]	4	9
User4 | [Free]	4	1	2	19 [18+1]	14	15
User5 | [Priority]	6	4	1	15 [11+4]	5	9

Average WT: [(0+14+4+14+5)/5] = **7.4** | **Average TAT**: [(6+17+9+15+9)/5] = **11.2**

Order of completion:

1 -> 3 -> 5 -> 2 -> 4

```
-- Shortest Job First ---

00:06  User1      [Member]   left after waiting   0s (server processed in  6s) | TAT:   6s | loaded at: 00:06
00:10  User2      [Member]   left after waiting   6s (server processed in  3s) | TAT:   9s | loaded at: 00:10
00:19  User3      [VIP]      left after waiting  12s (server processed in  5s) | TAT:  17s | loaded at: 00:19
00:07  User4      [Member]   left after waiting   2s (server processed in  1s) | TAT:   3s | loaded at: 00:07
00:14  User5      [VIP]      left after waiting   4s (server processed in  4s) | TAT:   8s | loaded at: 00:14

Avg Wait Time: 4.8s  |  Avg Turnaround Time: 8.6s  |  Avg Completion Time: 11.2s
```

Process (Unique User)	**Arrival**	**Burst**	**CT**	**WT: TAT-BT**	**TAT: CT-AT**
User1 | [Free]	0	6	6 [6+0]	0	6
User2 | [Free]	1	3	10 [7+3]	6	9
User3 | [Priority]	2	5	19 [14+5]	12	17
User4 | [Free]	4	1	7 [6+1]	2	3
User5 | [Priority]	6	4	14 [10+4]	4	8

Order of completion:

1 -> 4 -> 2 -> 5 -> 3

Average WT: [(0+6+12+2+4)/5] = **4.8** | **Average TAT**: [(6+9+17+3+8)/5] = **8.6**

Comparisons

Algorithm	FCFS	SJF	Priority
Avg Wait Time	6.2s	4.8s	7.4s
Avg Turnaround	10.0s	8.6s	11.2s
Avg Completion	12.6s	11.2s	13.8s

FCFS

- Players with longer burst times create a bottleneck for everyone who arrives after
- Most fair
- Simple to implement, but least efficient

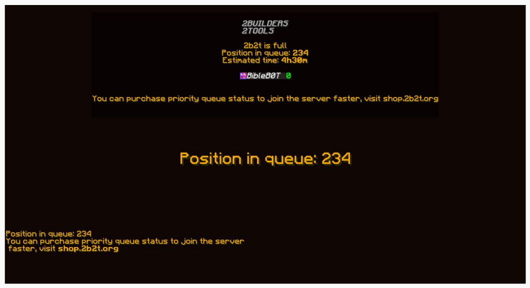
SJF

- Best system wide performance
- Best averages
- Players with short connection times get in faster
- Unfair to slow-connecting players (User3)

Priority

- Worst system wide averages
- Best experience for paying users (VIP)
- User3 and User5 waited only 4-5s
- User2 and User4 waited 14s each (longest wait time of any algo)

Key Insights



First Come First Serve (FCFS)

- Avg Wait Time: 6.2s
- Avg Turnaround Time: 10.0s
- Avg Completion Time: 12.6s

Shortest Job First (SJF)

- Avg Wait Time: 4.8s
- Avg Turnaround Time: 8.6s
- Avg Completion Time: 11.2s

Priority Scheduling

- Avg Wait Time: 7.4s
- Avg Turnaround Time: 11.2s
- Avg Completion Time: 13.8s

- Applying to the server for different needs:
  - FCFS: Retain fairness and enjoyment for all players and simplicity in implementation
  - SJF: Retain the best performance in terms of speed in processing player connections
  - Priority: Incentivize players from a monetary standpoint to join the game sooner
- Order of players will change depending on the algorithm

Conclusions

- None of these algorithms are necessarily better overall than one another (All excel in different areas and for different reasons)
    - FCFS is the most simple and fair to players, but may end up taking longer depending on their authentication time
    - SJF is most efficient in performance and time, but is prone to neglecting players that have longer connection time
    - Priority ensures that performance is efficient for prioritized players, but is unfair to players that aren’t prioritized and may even take longer based on the premium members’ connection times.
- Ideal to maintain the game server with a hybrid of at least two these algorithms that can perform reasonably well and be fair to all players regardless of their status.